

BAX 423 Homework 1 Part 1

Logan Garcia, Bonnie Hines

1. You're building a song recommendation tool. List why and why not you might want to pick a "frontier model", a proprietary model that is the most advanced, versus an open source model.

Open source is the default because cost and problem size line up. People do not need the most advanced proprietary model just to pick a next song, and after a point you pay more and maintain more without better recommendations. A frontier API on a narrow rec feature often means integration work and vendor churn instead of a clear jump in quality.

A frontier model is reasonable when you need help with compliance, huge scale, or a model you cannot ship in house, or when you want the vendor to run the service. The tradeoffs are usage-based cost, dependence on their roadmap, and possibly sending listening data outside your stack.

If you self-host an open model, data stays in your environment and cost looks like compute plus engineers. You are responsible for training, releases, and incidents. At Spotify scale that trade usually wins until there is a strong reason to pay for a frontier API.

2. Now assume you're building the model for Spotify and you have access to users' listening histories. What are some features you might want to compute? When you move from training your model to serving will each of these features require streaming data?

You might compute plays by artist or genre, skip rate versus finish rate, time of day and day of week, the last few tracks in the session, a short taste vector from recent plays, and device or region when you collect those fields legitimately.

Training reads scheduled batches built from historical logs, so the training step itself does not need a live streaming pipeline attached.

In serving, slowly changing values such as daily genre counts can land in Cloudflare D1 after you refresh them on a schedule. For the feature reads that users request through the API most often, Cloudflare Cache can return a cached response so those calls stay fast instead of running a full API or origin round trip on every request. Values that describe the current moment, such as the playing track and recent skips in this session, still need to update in near real time while playback is happening, so that part of the feature path looks more like streaming even when the rest reads from Cloudflare. Training still leans on batch data, and production mixes those stored snapshots with inputs that change every few seconds during a session.

3. How does the requirements of your system change if you're sending users a list of recommendations that updates once a day/week (a "For you" playlist, essentially) versus if you are recommending a new song related to each song a user is currently listening to?

When the playlist updates once a day or week, you can run a heavier job overnight, cache the full list, and retry failures without stopping playback. Your bottleneck is how many batch runs you complete per day, not how fast each API call returns.

When you recommend on every track during playback, you are on a latency-sensitive path. You need fast responses, small feature lookups, caching, and often a smaller model so the service stays stable. Reliability and rate limits matter more than in the offline playlist case, and you spend more engineering on real-time data paths and low-latency storage.

4. Some artists become incredibly popular suddenly. New subgenres can emerge. What can you do to minimize the drift of your model?

Rely on production metrics such as skips, completes, and satisfaction on new and breakout tracks, not only a single offline accuracy number, because those metrics shift earlier than one aggregate score on a dashboard.

You should retrain often enough that training data reflects current taste. When you fit the model, let recent listening count for more than years-old history so the model tracks what people do now. Add fallbacks for cold tracks using metadata, audio, or text so new music is not invisible next to a model that only saw old co-listening patterns. Roll changes to a subset of users first, keep some randomness so new music still gets shown, and review behavior on new catalog after each release instead of trusting the schedule alone.